

JUSTICEFLOW

Fully Dressed Use Case Descriptions

AI-Powered Police Case & Evidence Management System

CLI Application · Ubuntu Linux 24.04 LTS · PostgreSQL 16 · C++ / Python 3.12

Modules Documented

- UC 1 — Login and Establish Officer Session (Authentication & Session Module — Shared Infrastructure)
- UC 2 — Log Database Operation and Enforce Audit Integrity (Audit & Compliance — Subsystem 3)
- UC 3 — Request, Execute, and Manage Warrants, Arrests, and Bail (Subsystem 3)
- UC 4 — Create, Authorize, and Deliver a Forensic Examination Request (Forensic & Lab — Subsystem 3)
- UC 5 — Trigger Workload Analysis and Generate Assignment Suggestions (Hungarian Algorithm AI — Subsystem 3)

Use Case 1 — Login and Establish Officer Session

Use Case 1: Login and Establish Officer Session	
Module	Authentication & Session Module Shared Infrastructure
Environment	justiceflow_main — the C++ CLI application running on Ubuntu Linux 24.04 LTS. The officer interacts via a terminal prompt. The application connects to PostgreSQL 16 via a local Unix domain socket (/var/run/postgresql/.s.PGSQL.5432) using libpq. Session state is stored in a POSIX shared memory segment maintained by the justiceflow_scheduler daemon and in PostgreSQL session variables (app.current_officer_id, app.current_belt_number) used by the audit trigger layer.
Level	Essential, main user-goal level. Every officer interaction with JusticeFlow begins with this use case. It is the mandatory security gateway before any module — case management, evidence, warrants, forensics, or AI dashboards — becomes accessible.
Primary Actor	Officer (any rank: CONSTABLE through IGP)
Secondary Actor	justiceflow_main (C++ application process)
Extra / Offstage Actors	PostgreSQL 16 (credential and duty roster validation), justiceflow_scheduler daemon (shared memory status table), audit.log_change() trigger (login event recording)
Stakeholders & Interests	Officer • Wants fast, reliable CLI login from any terminal on the station network • Wants clear, actionable error messages when login fails • Wants session to remain active for the full duty shift without repeat authentication SHO (Inspector / Station House Officer) • Wants assurance that only officers on the current duty roster can access the system • Wants suspended and retired officer CNICs automatically blocked at login time • Wants all login attempts — successful and failed — recorded in the immutable audit log System / Database (justice_app role) • Wants the PostgreSQL session variable app.current_officer_id injected immediately after authentication so every subsequent INSERT/UPDATE/DELETE is attributed to the correct officer in the audit trail • Wants the officer rank stored in session context so rank-validation triggers (validate_fir_filing_rank, validate_warrant_request_rank) can fire correctly on subsequent operations Department / Legal Authority • Wants a tamper-proof record that every session was opened by a real, authorized, on-duty officer • Wants compliance with Pakistani CrPC access-control obligations
Preconditions	1. justiceflow_main is running and listening on the terminal

	2. PostgreSQL 16 is running and accepting Unix domain socket connections from the justice_app role 3. The officer has a registered row in the Officers table with a valid CNIC (15-digit format) and status = ACTIVE 4. The officer has a row in the Duty_Roster table with duty_status = SCHEDULED or ON_DUTY for the current date and shift 5. The POSIX shared memory segment for the agent status table is initialized by justiceflow_scheduler																		
Guarantee (Postcondition)	Minimum Guarantee (Failure): No PostgreSQL session variable is set — all subsequent DB operations will fail the audit trigger's officer-identity check No session entry is created in the in-memory session registry (protected by pthread_mutex_t) The failed login attempt is written to audit.Audit_Log via the audit trigger (action = INSERT on the login_attempts tracking record) The connection semaphore slot (sem_t) is released immediately on failure Success Guarantee: PostgreSQL session variables app.current_officer_id and app.current_belt_number are set for the connection Officer rank, station_id, and assigned shift are loaded into the in-memory session struct (pinned in physical RAM via mlock()) A session entry is registered in the shared session registry (pthread_mutex_t protected) Successful login event is captured in audit.Audit_Log with officer CNIC, belt number, pg_backend_pid(), client process UID, and timestamp Officer is presented with their rank-appropriate CLI menu																		
Success Case	Officer enters CNIC and password at the CLI prompt. The system validates credentials, confirms active duty status, injects session context into PostgreSQL, and presents the officer with their authorized command menu.																		
Trigger	Officer runs justiceflow_main at the terminal and is presented with the login prompt, then enters their 15-digit CNIC and password.																		
Sequence of Steps	<table><tr><th>Step</th><th>Actor</th><th>Activity</th></tr><tr><td>1</td><td>Officer</td><td>Enters 15-digit CNIC and password at the CLI login prompt</td></tr><tr><td>2</td><td>justiceflow_main</td><td>Validates CNIC format (15-digit, numeric) and non-empty password client-side before sending to DB</td></tr><tr><td>3</td><td>justiceflow_main (libpq)</td><td>Opens connection to PostgreSQL via Unix domain socket as justice_app role</td></tr><tr><td>4</td><td>PostgreSQL</td><td>Executes credential check query against Officers table; returns officer_id, rank, station_id, status, belt_number</td></tr><tr><td>5</td><td>PostgreSQL (validate_officer_duty_status trigger)</td><td>Checks Duty_Roster for the officer's scheduled entry today; rejects if absent or not on roster</td></tr></table>	Step	Actor	Activity	1	Officer	Enters 15-digit CNIC and password at the CLI login prompt	2	justiceflow_main	Validates CNIC format (15-digit, numeric) and non-empty password client-side before sending to DB	3	justiceflow_main (libpq)	Opens connection to PostgreSQL via Unix domain socket as justice_app role	4	PostgreSQL	Executes credential check query against Officers table; returns officer_id, rank, station_id, status, belt_number	5	PostgreSQL (validate_officer_duty_status trigger)	Checks Duty_Roster for the officer's scheduled entry today; rejects if absent or not on roster
Step	Actor	Activity																	
1	Officer	Enters 15-digit CNIC and password at the CLI login prompt																	
2	justiceflow_main	Validates CNIC format (15-digit, numeric) and non-empty password client-side before sending to DB																	
3	justiceflow_main (libpq)	Opens connection to PostgreSQL via Unix domain socket as justice_app role																	
4	PostgreSQL	Executes credential check query against Officers table; returns officer_id, rank, station_id, status, belt_number																	
5	PostgreSQL (validate_officer_duty_status trigger)	Checks Duty_Roster for the officer's scheduled entry today; rejects if absent or not on roster																	

	6	PostgreSQL	Checks officer_status_enum: rejects if SUSPENDED, ON_LEAVE, RETIRED, or TERMINATED
	7	justiceflow_main	Executes SET SESSION app.current_officer_id = '...' and SET SESSION app.current_belt_number = '...' on the authenticated connection
	8	justiceflow_main	Loads officer rank and station into the in-memory session struct; registers session in the pthread_mutex_t-protected session registry
	9	audit.log_change() trigger	Writes login event to audit.Audit_Log capturing officer identity, pg_backend_pid(), client IP from PostgreSQL connection metadata, and timestamp
	10	justiceflow_main	Renders the rank-appropriate CLI menu (e.g., SHO menu includes warrant approval and case closure; Constable menu is read-only)
Extensions (Alternative / Exception Flows)	2a — Invalid CNIC format justiceflow_main displays: 'Invalid CNIC — must be 15 numeric digits.' No DB connection is opened. Officer is re-prompted. Failed attempt counter incremented in local memory. 4a — CNIC not found or password mismatch PostgreSQL returns zero rows. justiceflow_main displays: 'Authentication failed. Invalid CNIC or password.' Failed attempt is logged to audit.Audit_Log. After 3 consecutive failures: justiceflow_main locks the CLI session for 15 minutes and logs a security alert. 6a — Officer status is SUSPENDED PostgreSQL status check returns SUSPENDED. validate_officer_duty_status fires and raises an exception. justiceflow_main displays: 'Access denied — account suspended. Contact station administration.' Attempt is logged in audit.Audit_Log with reason = SUSPENDED. 6b — Officer status is RETIRED or TERMINATED justiceflow_main displays: 'Access denied — account is no longer active.' Attempt logged. No session created. 5a — Officer not on today's duty roster Duty_Roster check finds no SCHEDULED or ON_DUTY row for today. justiceflow_main displays: 'Access denied — you are not scheduled for duty at this time. Contact your SHO.' Attempt logged. 3a — PostgreSQL Unix socket unavailable libpq PQconnectdb() returns CONNECTION_BAD. justiceflow_main displays: 'Database connection failed. Contact system administrator.' and exits gracefully.		

Technology & Platform

C++ (GCC 13) — justiceflow_main CLI application on Ubuntu Linux 24.04 LTS
libpq 16 — Unix domain socket connection to PostgreSQL (AF_UNIX, /var/run/postgresql/.s.PGSQL.5432)
PostgreSQL 16 — Officers, Duty_Roster tables; session variable injection via SET SESSION
pthread_mutex_t — session registry mutual exclusion
sem_t (counting semaphore) — max 50 concurrent session enforcement
mlock() — session struct pinned in physical RAM (no swap)
audit.log_change() SECURITY DEFINER trigger — immutable login event recording
POSIX shared memory (shm_open + mmap) — real-time session status visible to justiceflow_scheduler

Use Case 2 — Log Database Operation and Enforce Audit Integrity

Use Case 2: Log Database Operation and Enforce Audit Integrity	
Module	Audit & Compliance Module Subsystem 3
Environment	PostgreSQL 16 trigger layer — audit.log_change() SECURITY DEFINER function attached via AFTER triggers to all 8 audited tables. Operates entirely inside the database engine process. Invoked automatically on every INSERT, UPDATE, or DELETE issued by the justice_app role from the C++ application. Audit records are written to the append-only audit.Audit_Log table. The audit schema is completely inaccessible to both justice_app (write) and justice_ai (any access) at the database permission level.
Level	Essential, supporting/system-goal level. This use case is non-interactive and requires no officer action. It fires automatically as a side-effect of every data modification operation across the system.
Primary Actor	System (PostgreSQL trigger engine — audit.log_change() function)
Secondary Actor	Auditor (queries audit log post-facto)
Extra / Offstage Actors	SHO (queries change history for cases under command), Court / Legal Authority (relies on audit trail as chain-of-custody evidence), justiceflow_main C++ process (provides session context via SET SESSION variables)
Stakeholders & Interests	System / Database Engine • Wants every INSERT, UPDATE, and DELETE on audited tables captured atomically in the same transaction as the originating operation • Wants BEFORE triggers to block all destructive operations on evidence and audit records before they reach disk • Wants audit.log_change() to run as SECURITY DEFINER so justice_app cannot accidentally or maliciously suppress an audit write Auditor • Wants searchable audit records filterable by officer belt number, case FIR number, table name, action type, and time window • Wants JSONB snapshots of old_data and new_data in every record so the exact change is reconstructable without application logic • Wants certainty that no audit record has ever been modified or deleted SHO • Wants to query the full change history for any case under their station • Wants to identify which officer made a disputed change and when Court / Legal Authority • Wants a provably immutable chain of custody for all evidence operations • Wants the audit trail admissible as digital evidence under Pakistani cyber-crime and evidence laws
Preconditions	1. Officer is authenticated and PostgreSQL session variables app.current_officer_id and app.current_belt_number are set on the connection 2. A data modification statement (INSERT / UPDATE / DELETE) has been issued on one of the 8 audited tables: Cases, Evidence, Officers, Arrests, Warrants, charge_sheets, Bail_Records, Accused

	3. audit.Audit_Log table is online and accepting inserts via the SECURITY DEFINER trigger function 4. The AFTER trigger trg_audit_<table> is enabled on the target table																					
Guarantee (Postcondition)	Minimum Guarantee (Failure): • If the originating operation is rolled back (e.g., due to a validation trigger raising an exception), the audit write is also rolled back — no phantom audit records are created • Under no circumstances can any user directly INSERT, UPDATE, or DELETE rows in audit.Audit_Log — the permission model (REVOKE ALL ON SCHEMA audit FROM PUBLIC; no GRANT to justice_app or justice_ai) enforces this at the database engine level Success Guarantee: • An immutable row is appended to audit.Audit_Log containing: audited_table (enum), record_id (PK of changed row), action (INSERT/UPDATE/DELETE enum), old_data (JSONB full before-snapshot), new_data (JSONB full after-snapshot), changed_by_db_user (justice_app), officer_id and belt_number from session variables, pg_backend_pid() (OS process ID of the C++ connection), client_ip from PostgreSQL connection metadata, changed_at (timestampz) • All four immutability blocks (evidence hard-delete, audit-log delete, audit-log update, locked charge-sheet update) remain permanently enforced via BEFORE triggers that raise EXCEPTION and abort the operation																					
Success Case	Every data modification on an audited table automatically produces a complete, tamper-proof audit record attributing the change to the exact officer, process, and IP — with no application-layer involvement required.																					
Trigger	PostgreSQL executes an INSERT, UPDATE, or DELETE statement on any of the 8 audited tables within the justice_app connection.																					
Sequence of Steps	<table><tr><th>Step</th><th>Actor</th><th>Activity</th></tr><tr><td>1</td><td>justiceflow_main (C++ / libpq)</td><td>Executes a parameterized SQL statement (INSERT/UPDATE/DELETE) on an audited table via the justice_app role connection</td></tr><tr><td>2</td><td>PostgreSQL trigger engine</td><td>Fires the BEFORE trigger(s) for the table (number generators, validation, immutability checks) — operation is aborted here if any BEFORE trigger raises EXCEPTION</td></tr><tr><td>3</td><td>PostgreSQL</td><td>Writes the row to disk (transaction still open)</td></tr><tr><td>4</td><td>PostgreSQL trigger engine</td><td>Fires trg_audit_<table> AFTER trigger; calls audit.log_change() as SECURITY DEFINER</td></tr><tr><td>5</td><td>audit.log_change()</td><td>Reads TG_OP (INSERT/UPDATE/DELETE), TG_TABLE_NAME, and the OLD/NEW row records from the trigger context</td></tr><tr><td>6</td><td>audit.log_change()</td><td>Reads current_setting('app.current_officer_id') and current_setting('app.current_belt_number') from the session variable set at login</td></tr></table>	Step	Actor	Activity	1	justiceflow_main (C++ / libpq)	Executes a parameterized SQL statement (INSERT/UPDATE/DELETE) on an audited table via the justice_app role connection	2	PostgreSQL trigger engine	Fires the BEFORE trigger(s) for the table (number generators, validation, immutability checks) — operation is aborted here if any BEFORE trigger raises EXCEPTION	3	PostgreSQL	Writes the row to disk (transaction still open)	4	PostgreSQL trigger engine	Fires trg_audit_<table> AFTER trigger; calls audit.log_change() as SECURITY DEFINER	5	audit.log_change()	Reads TG_OP (INSERT/UPDATE/DELETE), TG_TABLE_NAME, and the OLD/NEW row records from the trigger context	6	audit.log_change()	Reads current_setting('app.current_officer_id') and current_setting('app.current_belt_number') from the session variable set at login
Step	Actor	Activity																				
1	justiceflow_main (C++ / libpq)	Executes a parameterized SQL statement (INSERT/UPDATE/DELETE) on an audited table via the justice_app role connection																				
2	PostgreSQL trigger engine	Fires the BEFORE trigger(s) for the table (number generators, validation, immutability checks) — operation is aborted here if any BEFORE trigger raises EXCEPTION																				
3	PostgreSQL	Writes the row to disk (transaction still open)																				
4	PostgreSQL trigger engine	Fires trg_audit_<table> AFTER trigger; calls audit.log_change() as SECURITY DEFINER																				
5	audit.log_change()	Reads TG_OP (INSERT/UPDATE/DELETE), TG_TABLE_NAME, and the OLD/NEW row records from the trigger context																				
6	audit.log_change()	Reads current_setting('app.current_officer_id') and current_setting('app.current_belt_number') from the session variable set at login																				

	7	audit.log_change()	Reads pg_backend_pid() for the OS-level process ID of the C++ application connection
	8	audit.log_change()	INSERTs a new row into audit.Audit_Log with all captured fields as a single atomic write within the same transaction
	9	PostgreSQL	Transaction is committed — both the original data change and the audit record are durable simultaneously
	10	Auditor / SHO (post-facto)	Queries audit.Audit_Log filtered by officer_id, FIR number, time window, or action type via the CLI audit report command
Extensions (Alternative / Exception Flows)	2a — BEFORE trigger raises EXCEPTION (e.g., rank validation failure) Transaction is aborted. Row never reaches disk. Audit record is also rolled back — no phantom record for an operation that never completed. The exception message is returned to justiceflow_main which displays it to the officer at the CLI.		
	2b — Evidence hard-delete attempted (prevent_evidence_hard_delete trigger) BEFORE DELETE trigger on Evidence table raises: 'Hard delete of evidence is prohibited. Use soft delete (is_deleted = TRUE).'		
	DELETE statement is aborted. Evidence row remains intact. Attempt is still captured in audit.Audit_Log because the BEFORE immutability trigger fires and raises — but the AFTER audit trigger does not fire for the blocked operation. The block itself is logged by the system error log.		
	2c — Audit log direct-delete attempted (prevent_audit_log_delete trigger) BEFORE DELETE on audit.Audit_Log raises: 'Audit log records are immutable and cannot be deleted.'		
	Operation is aborted. Attempt is flagged in the PostgreSQL server log.		
Technology & Platform	2d — Locked charge sheet update attempted (prevent_locked_charge_sheet_update trigger) BEFORE UPDATE trigger checks is_locked = TRUE. If locked, raises: 'Charge sheet is locked and cannot be modified after court submission.'		
	Update is rejected. Charge sheet remains unchanged.		
	6a — Session variable not set (officer not authenticated properly) current_setting('app.current_officer_id') returns empty string or raises an error.		
	audit.log_change() logs changed_by_officer_id as NULL with a warning flag. Security alert is raised and the connection is flagged for administrator review — this should never occur in normal operation because the login use case always sets the session variable before any other operation is permitted.		
	PostgreSQL 16 — AFTER triggers on 8 tables; audit.log_change() SECURITY DEFINER PL/pgSQL function audit schema — REVOKE ALL FROM PUBLIC; no direct write access for any application role justice_app role — SELECT/INSERT/UPDATE/DELETE on public schema only justice_ai role — SELECT on public schema only; NO ACCESS to audit schema		

POSIX process model — `pg_backend_pid()` captures the OS-level PID of the C++ `justiceflow_main` connection

JSONB — full row snapshots stored as `old_data` / `new_data` for complete change reconstruction

`pthread_mutex_t` — protects in-memory session registry that maps connections to officers

`idx_audit_log_table_record`, `idx_audit_log_changed_at`,
`idx_audit_log_officer_id` — partial indexes for fast audit queries

Use Case 3 — Request, Execute, and Manage Warrants, Arrests, and Bail

Use Case 3: Request, Execute, and Manage Warrants, Arrests, and Bail	
Module	Warrant, Arrest & Bail Module Subsystem 3
Environment	justiceflow_main C++ CLI application on Ubuntu Linux 24.04 LTS. Officers interact via terminal menu commands. The application connects to PostgreSQL 16 via Unix domain socket using libpq. Automated expiry of overdue warrants and bail records is handled by the justiceflow_scheduler daemon process using SIGALRM-driven timer scheduling — it calls expire_warrants() and expire_bail_records() PostgreSQL stored functions daily at 00:01 via a direct libpq SQL call, with no dependency on external cron infrastructure.
Level	Essential, main user-goal level. Covers the complete legal lifecycle of a warrant from issuance through execution or cancellation, the full arrest workflow with custody status management, and bail recording and revocation.
Primary Actor	SHO (INSPECTOR rank — station_type context: POLICE_STATION)
Secondary Actor	Investigation Officer (SIO / IO), Executing Officer, Recording Officer, Court
Extra / Offstage Actors	justiceflow_scheduler daemon (warrant and bail expiry jobs), PostgreSQL trg_generate_warrant_number / trg_generate_arrest_number / trg_generate_bail_number triggers (auto-numbering), audit.log_change() trigger (full audit trail on Warrants, Arrests, Bail_Records tables)
Stakeholders & Interests	SHO (Inspector) - Wants to request all three warrant types (ARREST, SEARCH, SEIZURE) from the CLI with automatic warrant number generation — no manual numbering - Wants to cancel warrants and revoke bail records with reason documentation - Wants the system to automatically expire overdue warrants at 00:01 daily without any manual intervention Executing Officer - Wants to see only warrants assigned to them in a clear CLI list before executing - Wants to record arrest details (suspect CNIC cross-referenced against Persons and Accused tables) with auto-generated ARR-YYYY-STATIONCODE-NNNN number - Wants to update custody_status_enum without ambiguity Recording Officer / Court - Wants bail records linked to the correct arrest_id and case FIR number with auto-generated BAIL-YYYY-STATIONCODE-NNNN number - Wants bail_type_enum (REGULAR, ANTICIPATORY, INTERIM, SURETY) properly recorded - Wants bail revocation with documented reason stored in the record Legal / Compliance Authority - Wants warrant_status_enum transitions (ISSUED → EXECUTED / CANCELLED / EXPIRED) to be the only permitted state changes

	• Wants every operation on Warrants, Arrests, and Bail_Records tables captured in audit.Audit_Log • Wants automatic expiry of stale warrants (valid_until < NOW()) to prevent legally invalid instruments from remaining in ISSUED status						
Preconditions	1. Officer is authenticated and the PostgreSQL session variables are set (app.current_officer_id, app.current_belt_number) 2. An active case (case_status_enum != CLOSED) exists in the Cases table to which the warrant/arrest/bail will be linked 3. For warrant requests: officer rank must satisfy validate_warrant_request_rank() — INSPECTOR or above 4. For arrest recording: an executed warrant must exist for the suspect, or the executing officer has authority for warrantless arrest under relevant Pakistani law 5. justiceflow_scheduler daemon is running and the daily expiry jobs are scheduled via SIGALRM timer						
Guarantee (Postcondition)	Minimum Guarantee (Failure): • No warrant_number is consumed from Sequence_Registry if the warrant INSERT fails or is rolled back • No arrest_number or bail_number is issued for an incomplete or failed workflow • No custody status change is persisted if the update transaction is rolled back • All failed attempts are captured in audit.Audit_Log via the AFTER trigger Success Guarantee: • Warrant row created with auto-generated WRT-YYYY-STATIONCODE-NNNN number, linked to the correct case FIR, warrant_status = ISSUED, valid_until set • Arrest row created with auto-generated ARR-YYYY-STATIONCODE-NNNN, linked to case and warrant, custody_status = IN_CUSTODY • Bail row created with auto-generated BAIL-YYYY-STATIONCODE-NNNN, linked to arrest and case, bail_status = ACTIVE • All three records fully captured in audit.Audit_Log (trg_audit_warrants, trg_audit_arrests, trg_audit_bail_records) • Overdue warrants and bail records automatically set to EXPIRED / EXPIRED status daily at 00:01 by justiceflow_scheduler calling expire_warrants() and expire_bail_records() stored functions						
Success Case	SHO requests an arrest warrant via CLI. The system auto-generates a warrant number and persists the record. The executing officer records the arrest. The recording officer or court records bail. The scheduler automatically expires any overdue records at the daily scheduled time.						
Trigger	Authenticated SHO selects 'Request Warrant' from the warrant operations CLI menu, or Executing Officer selects 'Record Arrest', or Recording Officer selects 'Record Bail'.						
Sequence of Steps	<table><tr><th>Step</th><th>Actor</th><th>Activity</th></tr><tr><td>1</td><td>SHO</td><td>Selects warrant type from CLI menu (ARREST / SEARCH / SEIZURE) and enters case FIR</td></tr></table>	Step	Actor	Activity	1	SHO	Selects warrant type from CLI menu (ARREST / SEARCH / SEIZURE) and enters case FIR
Step	Actor	Activity					
1	SHO	Selects warrant type from CLI menu (ARREST / SEARCH / SEIZURE) and enters case FIR					

		number, suspect details, and valid_until date
2	<i>justiceflow_main (libpq)</i>	Executes parameterized INSERT INTO Warrants(...) via justice_app role connection
3	<i>PostgreSQL (trg_validate_warrant_request_rank)</i>	BEFORE INSERT trigger fires; confirms requesting officer is INSPECTOR or above — raises EXCEPTION if not
4	<i>PostgreSQL (trg_generate_warrant_number)</i>	BEFORE INSERT trigger fires; calls get_next_seq() for atomic collision-free numbering; assigns WRT-YYYY-STATIONCODE-NNNN to NEW.warrant_number
5	<i>PostgreSQL (trg_audit_warrants)</i>	AFTER INSERT trigger fires; audit.log_change() writes full warrant snapshot to audit.Audit_Log
6	<i>Executing Officer</i>	Views warrant list at CLI; selects warrant to execute in the field; system updates warrant_status to EXECUTED
7	<i>Executing Officer</i>	Selects 'Record Arrest' from CLI; enters suspect CNIC (cross-referenced against Persons and Accused tables), location, time of arrest
8	<i>PostgreSQL (trg_generate_arrest_number)</i>	BEFORE INSERT generates ARR-YYYY-STATIONCODE-NNNN; trg_audit_arrests writes to audit.Audit_Log
9	<i>Executing Officer</i>	Updates custody_status_enum to IN_CUSTODY; updates Case_Status_Log entry for the arrest event
10	<i>Recording Officer / Court</i>	Selects 'Record Bail' from CLI; enters bail_type_enum, bail amount, surety details, valid_until; links to arrest_id
11	<i>PostgreSQL (trg_generate_bail_number)</i>	BEFORE INSERT generates BAIL-YYYY-STATIONCODE-NNNN; trg_audit_bail_records writes to audit.Audit_Log
12	<i>justiceflow_scheduler daemon</i>	At 00:01 daily: SIGALRM fires timer tick; scheduler calls expire_warrants() stored function

		via libpq — sets all ISSUED warrants past valid_until to EXPIRED; calls expire_bail_records() — sets all ACTIVE bail records past valid_until to EXPIRED; both functions emit RAISE NOTICE with affected row count to the scheduler's named FIFO for status reporting
Extensions (Alternative / Exception Flows)	3a — validate_warrant_request_rank() rejects the officer BEFORE INSERT trigger raises: 'Insufficient rank to request a warrant. INSPECTOR or above required.' INSERT is aborted. No warrant_number consumed. justiceflow_main displays the error at CLI. 3b — Duplicate active warrant detected for same suspect and case Application-layer check finds an existing ISSUED warrant for the same accused_cnic and case_id. CLI displays: 'An active warrant already exists for this suspect on this case (WRT-XXXX). Cancel it before creating a new one.' Officer may proceed to cancel the existing warrant first. 6a — SHO cancels warrant before execution SHO selects 'Cancel Warrant' from CLI menu; enters warrant_number and cancellation reason. UPDATE Warrants SET warrant_status = 'CANCELLED', cancellation_reason = '...' WHERE warrant_id = ...; trg_audit_warrants AFTER UPDATE fires; cancellation is fully recorded in audit.Audit_Log. Executing officer notified via the shared memory status table (next CLI status refresh). 9a — Executing officer marks arrest as disputed Executing Officer selects 'Mark Arrest as Disputed'; enters dispute reason. UPDATE Arrests SET is_disputed = TRUE, dispute_reason = '...' — triggers audit write. Case is flagged in the CLI SHO dashboard with a DISPUTED indicator for review. 10a — SHO revokes bail SHO selects 'Revoke Bail' from CLI; enters bail_number and revocation reason (bail conditions violated). UPDATE Bail_Records SET bail_status = 'REVOKED', revocation_reason = '...' Suspect's custody_status in Arrests updated to IN_CUSTODY. Full audit trail written by trg_audit_bail_records. 12a — expire_warrants() affects zero rows Function emits RAISE NOTICE: 'expire_warrants(): 0 warrants expired.' to the named FIFO. Scheduler reads status from named FIFO and updates shared memory agent status table. No error condition.	
Technology & Platform	C++ (GCC 13) — justiceflow_main CLI on Ubuntu Linux 24.04 LTS libpq 16 — parameterized queries via Unix domain socket	

PostgreSQL 16 — Warrants, Arrests, Bail_Records tables with BEFORE/AFTER trigger chains
PL/pgSQL stored functions — expire_warrants(), expire_bail_records(), get_next_seq() atomic upsert
justiceflow_scheduler daemon — double-fork pattern, SIGALRM + setitimer(), daily 00:01 legal expiry jobs
Named FIFOs (mkfifo) — scheduler reads expiry job completion status from AI/stored-function output pipes
SIGCHLD handler + waitpid(WNOHANG) — zombie-free child process cleanup for any forked expiry sub-processes
audit.Audit_Log — trg_audit_warrants, trg_audit_arrests, trg_audit_bail_records (AFTER triggers)
idx_warrants_expiry_check, idx_bail_records_expiry_check — partial indexes on ISSUED/ACTIVE records only for fast nightly expiry query

Use Case 4 — Create, Authorize, and Deliver a Forensic Examination Request

Use Case 4: Create, Authorize, and Deliver a Forensic Examination Request	
Module	Forensic & Lab Module Subsystem 3
Environment	justiceflow_main C++ CLI application on Ubuntu Linux 24.04 LTS. IO and SHO interact via terminal menu commands within an active case context. Evidence files (forensic images, scanned lab reports, CCTV stills) are accessed via mmap(MAP_PRIVATE, PROT_READ) memory-mapped I/O — the kernel pages evidence file content on demand without loading entire files into heap memory. The application connects to PostgreSQL 16 via Unix domain socket using libpq (justice_app role). State sync triggers (trg_sync_evidence_sent_to_lab, trg_sync_evidence_returned_from_lab) automatically maintain evidence_status_enum consistency without any application-layer coordination.
Level	Essential, main user-goal level. Represents the complete forensic evidence lifecycle from initial request creation through laboratory analysis to report delivery and optional contestation.
Primary Actor	IO (Investigating Officer — SIO or IO role in Case_Officers table)
Secondary Actor	SHO (authorizes the forensic request), System (state-sync triggers maintain evidence status automatically)
Extra / Offstage Actors	Forensic Laboratory (physical external actor — receives and processes evidence), Court (receives and may contest the forensic report), audit.log_change() trigger (records all operations on Forensic_Lab_Requests and Evidence tables)
Stakeholders & Interests	IO (Investigating Officer) • Wants to create a forensic request linked to a case FIR number from the CLI without filling in a warrant number or other unrelated data • Wants evidence items linked to the request with their evidence_number (EVD-FIR-XXXX format) automatically updated to SENT_TO_LAB status on dispatch — no manual UPDATE required • Wants to record lab receipt, examination findings, and deliver the final report from the CLI with all state transitions handled by triggers • Wants to record amendments or contest a report when findings appear incorrect SHO (Inspector) • Wants to review and authorize forensic requests from the CLI before evidence leaves the station • Wants assurance that evidence cannot be dispatched to the lab without explicit authorization (forsenic_request_status_enum must pass REQUESTED → AUTHORIZED before trg_sync_evidence_sent_to_lab fires) System (Trigger Layer) • trg_sync_evidence_sent_to_lab must fire on INSERT to Forensic_Request_Evidence to atomically update all linked evidence rows to evidence_status = SENT_TO_LAB • trg_sync_evidence_returned_from_lab must fire on UPDATE to Forensic_Lab_Requests when status transitions to REPORT_DELIVERED to atomically update all linked evidence rows to RETURNED_FROM_LAB • Both triggers must operate within the same transaction as the originating statement to maintain ACID consistency

	Court / Legal Authority • Wants a complete, auditable chain of custody for all evidence items: RECEIVED → SEALED → SENT_TO_LAB → RETURNED_FROM_LAB → PRODUCED_IN_COURT • Wants forensic reports stored with full metadata (examiner, examination_purpose_enum, findings, report date) for admissibility • Wants contested reports flagged (forsenic_request_status_enum = CONTESTED) and linked to the contestation reason for judicial review									
Preconditions	1. Officer (IO) is authenticated with a valid session and Case_Officers row with role_enum = SIO, IO, or LEAD_INVESTIGATOR for the target case 2. Target case exists and is in a modifiable status (UNDER_INVESTIGATION or EVIDENCE_COLLECTED) 3. At least one Evidence row linked to the case has evidence_status = SEALED (ready for dispatch) 4. SHO (Inspector-rank officer) is available to perform authorization at the CLI 5. Evidence files referenced in the request are present in the monitored evidence directory (watched by inotify-based file integrity monitor from the OS layer)									
Guarantee (Postcondition)	Minimum Guarantee (Failure): • If the forensic request INSERT fails (e.g., SHO authorization rejected), no Forensic_Request_Evidence rows are created and no evidence_status is changed • Evidence items remain in their current status (SEALED) if the request is rejected — no orphaned SENT_TO_LAB records exist without a corresponding authorized request • All failed and partial operations are captured in audit.Audit_Log Success Guarantee: • Forensic_Lab_Requests row created with auto-generated FLR-YYYY-STATIONCODE-NNNN number, forsenic_request_status_enum = AUTHORIZED • All linked Evidence rows atomically updated to evidence_status = SENT_TO_LAB by trg_sync_evidence_sent_to_lab (AFTER INSERT on Forensic_Request_Evidence) • Lab receipt, examination findings, and forensic report recorded and linked to the request • All linked Evidence rows atomically updated to evidence_status = RETURNED_FROM_LAB by trg_sync_evidence_returned_from_lab (AFTER UPDATE on Forensic_Lab_Requests when status = REPORT_DELIVERED) • Full audit trail written for every state transition on Forensic_Lab_Requests and Evidence tables									
Success Case	IO creates a forensic request, SHO authorizes it, evidence is dispatched and its status auto-updated, the lab returns findings, IO delivers the report, and all linked evidence is auto-marked as returned.									
Trigger	Authenticated IO selects 'Create Forensic Request' within an active case from the CLI evidence management menu.									
Sequence of Steps	<table><tr><th>Step</th><th>Actor</th><th>Activity</th></tr><tr><td>1</td><td>IO</td><td>Selects 'Create Forensic Request' from the CLI case menu; enters case FIR number, examination_purpose_enum (DNA / FINGERPRINT / DIGITAL / etc.), and request notes</td></tr><tr><td>2</td><td>justiceflow_main (libpq)</td><td>INSERT INTO Forensic_Lab_Requests(case_id,</td></tr></table>	Step	Actor	Activity	1	IO	Selects 'Create Forensic Request' from the CLI case menu; enters case FIR number, examination_purpose_enum (DNA / FINGERPRINT / DIGITAL / etc.), and request notes	2	justiceflow_main (libpq)	INSERT INTO Forensic_Lab_Requests(case_id,
Step	Actor	Activity								
1	IO	Selects 'Create Forensic Request' from the CLI case menu; enters case FIR number, examination_purpose_enum (DNA / FINGERPRINT / DIGITAL / etc.), and request notes								
2	justiceflow_main (libpq)	INSERT INTO Forensic_Lab_Requests(case_id,								

		requested_by, purpose, ...) with status = REQUESTED
3	PostgreSQL (trg_generate_forensic_request_number)	BEFORE INSERT generates FLR-YYYY-STATIONCODE-NNNN via get_next_seq()
4	SHO	Reviews pending forensic requests from the CLI authorization queue; selects the request and confirms authorization
5	justiceflow_main (libpq)	UPDATE Forensic_Lab_Requests SET status = 'AUTHORIZED', authorized_by = <sho_officer_id>
6	IO	Links specific evidence items to the request via CLI: selects evidence_number(s) from the case's sealed evidence list
7	justiceflow_main (libpq)	INSERT INTO Forensic_Request_Evidence(request_id, evidence_id) for each selected item
8	PostgreSQL (trg_sync_evidence_sent_to_lab)	AFTER INSERT on Forensic_Request_Evidence fires; UPDATE Evidence SET evidence_status = 'SENT_TO_LAB' WHERE evidence_id IN (linked items) — atomic within same transaction
9	IO	Records lab receipt confirmation once physical handover is complete: UPDATE Forensic_Lab_Requests SET lab_received_at = NOW(), status = 'RECEIVED_BY_LAB'
10	IO	Records examination findings returned from the laboratory: UPDATE Forensic_Lab_Requests SET findings = '...', examiner_name = '...', status = 'FINDINGS_RECORDED'
11	IO	Delivers the forensic report: UPDATE Forensic_Lab_Requests SET report_delivered_at = NOW(), status = 'REPORT_DELIVERED'
12	PostgreSQL (trg_sync_evidence_returned_from_lab)	AFTER UPDATE on Forensic_Lab_Requests (NEW.status = REPORT_DELIVERED) fires; UPDATE Evidence SET evidence_status = 'RETURNED_FROM_LAB' for all linked items — atomic

	13 <i>audit.log_change()</i> (<i>trg_audit_evidence</i>) All Evidence status changes are captured in audit.Audit_Log with old_data / new_data JSONB snapshots
Extensions (Alternative / Exception Flows)	4a — SHO rejects the forensic request SHO selects 'Reject' from the authorization queue and enters rejection reason. UPDATE Forensic_Lab_Requests SET status = 'REJECTED', rejection_reason = '...' No Forensic_Request_Evidence rows are created. Evidence remains SEALED. Audit record written. IO notified via CLI status display on next menu refresh. 6a — Evidence item is not in SEALED status (e.g., already SENT_TO_LAB for another request) Application-layer check rejects the linkage: 'Evidence EVD-XXXX is currently SENT_TO_LAB and cannot be linked to a new request.' IO must resolve the conflict (wait for return or select different evidence). 6b — Evidence file integrity violation detected by OS inotify monitor inotify monitor (OS layer) detects unexpected modification to an evidence file in the monitored directory. File is restored from backup and a security alert is written to the audit log and the system administrator is notified. Forensic request workflow is suspended pending investigation. 11a — IO records amendment to the delivered report IO selects 'Record Amendment' from the forensic request detail screen. UPDATE Forensic_Lab_Requests SET status = 'AMENDED', amendment_reason = '...', amended_findings = '...', amended_at = NOW() Original findings are preserved in old_data JSONB snapshot in audit.Audit_Log. Amendment is fully traceable. 11b — Court or IO contests the forensic report IO or SHO selects 'Contest Report' and enters contestation grounds. UPDATE Forensic_Lab_Requests SET status = 'CONTESTED', contestation_reason = '...' Case is flagged in the CLI dashboard with CONTESTED FORENSIC indicator. Re-examination workflow initiated.
Technology & Platform	C++ (GCC 13) — justiceflow_main CLI on Ubuntu Linux 24.04 LTS libpq 16 — parameterized DML via Unix domain socket PostgreSQL 16 — Forensic_Lab_Requests, Forensic_Request_Evidence, Evidence tables PL/pgSQL AFTER triggers — trg_sync_evidence_sent_to_lab, trg_sync_evidence_returned_from_lab (atomic state sync) trg_generate_forensic_request_number — BEFORE INSERT auto-numbering (FLR-YYYY-STATIONCODE-NNNN) mmap(MAP_PRIVATE, PROT_READ) + munmap() — memory-mapped access to large forensic evidence files (CCTV footage, forensic images) without heap allocation inotify (OS layer file integrity monitor) — watches evidence directory for unauthorized modification audit.Audit_Log — trg_audit_evidence, trg_audit_cases AFTER triggers; full JSONB old/new snapshots idx_forensic_requests_status, idx_forensic_request_evidence — partial indexes for fast request queue queries

Use Case 5 — Trigger Workload Analysis and Generate Officer Assignment Suggestions

Use Case 5: Trigger Workload Analysis and Generate Officer Assignment Suggestions	
Module	Officer Workload Balancer — Hungarian Algorithm Subsystem 3 (AI Layer)
Environment	justiceflow_scheduler daemon process (C++) on Ubuntu Linux 24.04 LTS — spawns workload_agent.py as a child process using fork() + execl(). workload_agent.py is a Python 3.12 script that connects to PostgreSQL 16 via psycopg2 using the justice_ai role (Unix domain socket, read-only on public schema, INSERT-only on analytics schema). The agent writes its output to analytics.Officer_Workload_Assignments. A named FIFO created with mkfifo() carries agent completion status back to the scheduler. The scheduler reads the FIFO asynchronously and updates the POSIX shared memory (shm_open + mmap) agent status table visible to justiceflow_main. SHO reviews and acts on suggestions from the CLI.
Level	Essential, supporting/system-goal level. This is a scheduled automated use case. The OS Scheduler is the only actor that can initiate a workload analysis run — no officer manually triggers it. SHO involvement is limited to reviewing and accepting or rejecting the suggestions produced.
Primary Actor	justiceflow_scheduler daemon (OS Scheduler)
Secondary Actor	SHO (reviews and acts on suggestions from CLI)
Extra / Offstage Actors	workload_agent.py (Python 3.12 process — executes the Hungarian Algorithm), justice_ai PostgreSQL role (read-only access to public schema, INSERT-only to analytics schema), expire_workload_assignments() stored function (called hourly by scheduler to expire stale SUGGESTED records)
Stakeholders & Interests	justiceflow_scheduler daemon • Wants workload_agent.py spawned via fork() + execl() every 6 hours with no zombie process accumulation (SIGCHLD handler + waitpid(WNOHANG)) • Wants the agent to write a status message (completion time, suggestion count, error code) to its named FIFO on completion • Wants the shared memory agent status table updated immediately after reading the FIFO so justiceflow_main can display live AI agent health on the CLI without a DB query SHO (Inspector) • Wants to see pending Officer_Workload_Assignments rows surfaced in a clear CLI review queue with officer name, current case count, and suggested case • Wants to accept or reject each suggestion individually — no bulk auto-apply without human authorization • Wants rejected suggestions to record a reason and wants no assignment changes from rejections Officers (Duty Officers / IOs) • Want workload distributed equitably — the Hungarian Algorithm minimizes standard deviation across the officer pool, not just total assignments

	- Want assignment suggestions to respect rank constraints (validate_officer_case_limit: max 10 active cases; validate_officer_duty_status: no assignment to suspended/on-leave officers) Department / Institution - Wants analytics.Model_Performance_Log rows written after every run recording: cost matrix dimensions, algorithm runtime, workload std_dev before and after, number of suggestions generated - Wants stale SUGGESTED assignments that were never actioned automatically expired hourly by expire_workload_assignments() to keep the suggestion queue clean
Preconditions	- justiceflow_scheduler daemon is running (initialized via double-fork pattern, setsid() for new session, /dev/null I/O redirect, all inherited file descriptors closed) - SIGALRM timer is armed via setitimer() to fire every 60 seconds; tick counter has reached the 6-hour threshold for the workload agent run - At least one active case (case_status_enum IN ('REGISTERED','UNDER_INVESTIGATION','EVIDENCE_COLLECTED')) exists in the Cases table - At least one ACTIVE officer with duty_status = ON_DUTY or SCHEDULED is present in the Duty_Roster for today - workload_agent.py is present at the configured path; Python 3.12 runtime is available - Named FIFO for workload_agent status reporting has been created with mkfifo() by the scheduler at startup
Guarantee (Postcondition)	Minimum Guarantee (Failure): - If workload_agent.py fails (non-zero exit code) or times out, the scheduler reads the error status from the named FIFO and updates the shared memory status table with agent_status = FAILED - No partial analytics.Officer_Workload_Assignments rows are written — the agent uses a transaction: all suggestions are INSERTed atomically or none are - No case-officer assignments in the public schema are modified — the agent has INSERT-only access to analytics schema and zero write access to public schema (enforced by justice_ai role permissions at the database level) - The failed run is logged to analytics.Model_Performance_Log with error details - The child process is reaped by the SIGCHLD handler via waitpid(WNOHANG) — no zombie process left in the process table Success Guarantee: - analytics.Officer_Workload_Assignments rows INSERTed with assignment_status = SUGGESTED, one row per recommended officer-case pairing, including confidence score and expires_at timestamp - analytics.Model_Performance_Log row INSERTed with: agent_name = 'workload_agent', run_timestamp, cost_matrix_dimensions, workload_std_dev_before, workload_std_dev_after, suggestion_count, runtime_ms - Named FIFO carries completion message: 'workload_agent: OK suggestions=N runtime=Xms' back to scheduler - Scheduler updates POSIX shared memory agent status table (pthread_mutex_t protected) to reflect successful run - SHO reviews and acts on suggestions at CLI; ACCEPTED suggestions trigger UPDATE on Case_Officers (via justice_app role); REJECTED suggestions record rejection_reason; SUGGESTED rows past expires_at are set to AUTO_EXPIRED by expire_workload_assignments() on the next hourly scheduler tick

Success Case	justiceflow_scheduler fires the 6-hour tick, forks workload_agent.py, the agent fetches case and officer data, builds the cost matrix, solves the Hungarian Algorithm, writes N assignment suggestions to the analytics schema, logs model performance, and reports completion via the named FIFO. SHO reviews suggestions at the CLI and accepts them, resulting in updated case-officer assignments.		
Trigger	justiceflow_scheduler daemon's SIGALRM tick counter reaches the 6-hour threshold for the workload agent run interval.		
Sequence of Steps	Step	Actor	Activity
	1	justiceflow_scheduler (SIGALRM handler)	Tick counter reaches 6-hour threshold; scheduler determines workload_agent.py must be spawned
	2	justiceflow_scheduler	Calls fork(); child process calls <code>execl('/usr/bin/python3', 'python3', '/opt/justiceflow/ai/workload_agent.py', NULL)</code> — child image replaced with Python runtime
	3	workload_agent.py (justice_ai role)	Connects to PostgreSQL via psycopg2 Unix domain socket as justice_ai role; fetches all active cases (case_id, case_type, filed_at, evidence_count, suspect_status) from public schema
	4	workload_agent.py	Fetches all available officers (officer_id, current_rank, active_case_count from Case_Officers, duty_status from Duty_Roster) — read-only SELECT on public schema
	5	workload_agent.py (NumPy)	Constructs cost matrix (officers × cases) using workload scoring: <code>cost[i][j] = f(officer_i.active_cases, case_j.priority_score, case_j.zone_risk)</code> via NumPy array operations
	6	workload_agent.py (SciPy)	Calls <code>scipy.optimize.linear_sum_assignment(cost_matrix)</code> — Hungarian Algorithm solves minimum-cost bipartite matching in $O(n^3)$ polynomial time; returns row_ind, col_ind optimal assignment arrays
	7	workload_agent.py (psycopg2)	Wraps all INSERTs in a single transaction; INSERTs one analytics.Officer_Workload_Assignments row per assignment: (officer_id, case_id, cost_score, assignment_status=SUGGESTED, suggested_at=NOW(), expires_at=NOW()+INTERVAL '1 hour')
	8	workload_agent.py (psycopg2)	INSERTs one analytics.Model_Performance_Log row with full run metrics; commits the transaction
	9	workload_agent.py	Writes status string to named FIFO: 'workload_agent: OK suggestions=N

		runtime=Xms std_dev_delta=-Y.YY'; exits with code 0
	10	<i>justiceflow_scheduler (SIGCHLD handler)</i> Receives SIGCHLD; calls waitpid(child_pid, WNOHANG) — reaps the completed Python child process; no zombie entry remains
	11	<i>justiceflow_scheduler</i> Reads status string from named FIFO; updates POSIX shared memory agent status table (pthread_mutex_t lock acquired, status written, lock released)
	12	<i>SHO (CLI)</i> Selects 'Review Workload Suggestions' from the CLI AI dashboard; justiceflow_main reads analytics.Officer_Workload_Assignments WHERE status = SUGGESTED and renders the queue
	13	<i>SHO</i> Accepts or rejects each suggestion individually from the CLI prompt
	14	<i>justiceflow_main (libpq, justice_app role)</i> For ACCEPTED: INSERT INTO Case_Officers(case_id, officer_id, role=IO, assigned_at=NOW()); UPDATE analytics.Officer_Workload_Assignments SET assignment_status = ACCEPTED. For REJECTED: UPDATE analytics.Officer_Workload_Assignments SET assignment_status = REJECTED, rejection_reason = '...' — no change to Case_Officers
Extensions (Alternative / Exception Flows)	3a — No active cases found in the database workload_agent.py fetches zero rows. Writes to named FIFO: 'workload_agent: NO_CASES suggestions=0' Exits with code 0. Scheduler updates shared memory: last_run=NOW(), status=NO_CASES. Model_Performance_Log row written with suggestion_count=0. No analytics.Officer_Workload_Assignments rows written.	
	4a — No available officers on duty today workload_agent.py fetches zero eligible officer rows. Writes: 'workload_agent: NO_OFFICERS suggestions=0' Scheduler updates shared memory. CLI displays 'No officers available — workload analysis skipped.' on next SHO dashboard refresh.	
	6a — scipy.optimize.linear_sum_assignment raises an exception workload_agent.py catches the exception; writes to FIFO: 'workload_agent: ERROR exception=<message>' Exits with non-zero code. No analytics INSERTs committed (transaction rolled back). SIGCHLD fires; scheduler reads FIFO error; updates shared memory status = FAILED. Scheduler logs alert. No zombie process (waitpid reaped the child).	
	9a — SHO rejects a suggestion SHO enters rejection reason at CLI prompt. UPDATE analytics.Officer_Workload_Assignments SET assignment_status = 'REJECTED', rejection_reason = '...'	

	Case_Officers table is NOT modified. The case remains in its current assignment state. 14a — SUGGESTED suggestion expires before SHO action justiceflow_scheduler's hourly tick calls expire_workload_assignments() stored function via libpq. UPDATE analytics.Officer_Workload_Assignments SET assignment_status = 'AUTO_EXPIRED' WHERE assignment_status = 'SUGGESTED' AND expires_at < NOW() Function emits RAISE NOTICE with affected row count to the named FIFO. Fresh suggestions will be generated at the next 6-hour workload agent run. 2a — workload_agent.py process times out (exceeds max_runtime threshold) Scheduler tracks child PID start time. If runtime exceeds configured maximum (e.g., 60 seconds), scheduler sends SIGTERM to child PID. Child process terminates; SIGCHLD fires; scheduler reaps with waitpid(WNOHANG). Timeout event logged to shared memory and Model_Performance_Log.
Technology & Platform	C++ (GCC 13) — justiceflow_scheduler daemon: double-fork pattern, setsid(), SIGALRM + setitimer(), fork() + execl() for agent spawning SIGCHLD signal handler + waitpid(WNOHANG) — zombie-free child process reaping; process registry mapping PIDs to agent types Named FIFO (mkfifo) — unidirectional status channel from workload_agent.py back to scheduler POSIX shared memory (shm_open + mmap) + pthread_mutex_t — real-time agent status table shared between scheduler and justiceflow_main Python 3.12 + psycpg2 2.9 — workload_agent.py connects to PostgreSQL via Unix domain socket as justice_ai role NumPy 1.26 — cost matrix construction via vectorized array operations SciPy 1.12 — scipy.optimize.linear_sum_assignment (Hungarian Algorithm, $O(n^3)$ optimal bipartite matching) PostgreSQL 16 — analytics.Officer_Workload_Assignments (INSERT-only by justice_ai), analytics.Model_Performance_Log, expire_workload_assignments() stored function idx_workload_expiry_check (partial on SUGGESTED) — fast hourly expiry scan justice_ai role permissions — SELECT on public schema; INSERT on analytics schema; NO ACCESS to audit schema; cannot UPDATE or DELETE in analytics (immutable prediction history) pthread_cond_t — worker threads in justiceflow_main notified when fresh AI results are available in shared memory (no busy-wait polling)